

1. ¿Qué problema resuelve publicar cambios en un repositorio remoto?

Publicar los cambios en GitHub hace que mi trabajo no quede solo en mi computador. Si mi PC se daña o la apago, los cambios siguen guardados en la nube. También permite que otras personas del equipo puedan ver lo que hice y trabajar sobre eso.

2. ¿Por qué es necesario traer cambios remotos antes de continuar trabajando?

Porque mientras yo estaba trabajando, otra persona pudo haber subido cambios al repositorio. Si no los traigo primero y sigo trabajando sobre una versión vieja, cuando intente subir mis cambios van a chocar con los de la otra persona y va a ser más difícil resolver eso. Es mejor siempre hacer git pull antes de empezar.

3. ¿Qué ventaja tiene reorganizar una rama antes de integrarla?

Cuando hice git rebase, mi rama quedó como si la hubiera creado desde la versión más reciente del proyecto. Eso hace que el historial se vea más ordenado y lineal, sin esas uniones raras que se crean cuando se hace merge sin actualizar primero. Es más fácil de leer y entender.

4. ¿Por qué se producen los conflictos?

Los conflictos pasan cuando dos personas modifican la misma parte del mismo archivo al mismo tiempo. Git no sabe cuál de las dos versiones dejar, entonces nos avisa y nos toca decidir nosotros cuál es la correcta o cómo combinar las dos.

5. ¿Cuál es el proceso lógico para resolver un conflicto correctamente?

Primero hay que abrir el archivo con conflicto y leer las dos versiones que Git marcó. Luego hay que entender qué quería hacer cada uno y decidir cómo queda mejor, ya sea eligiendo una versión o combinando las dos. Después se borran las marcas del conflicto, se guarda el archivo, se hace git add y se continúa con el proceso.

6. ¿Por qué conviene limpiar commits antes de hacer un Pull Request?

Porque cuando alguien va a revisar mi trabajo, lo primero que mira es el historial de commits. Si ve mensajes como "arreglo", "cambio" o "fix" no entiende nada de lo que hice. En cambio si los commits están limpios y con mensajes claros, la revisión es mucho más fácil y rápida.

7. ¿Qué ventaja tiene aplicar solo un cambio específico desde otra rama?

Con cherry-pick pude traer únicamente el cambio que estaba aprobado, sin traer los otros cambios que todavía no estaban listos. Eso es útil cuando en una rama experimental hay cosas a medias y solo una parte ya está lista para entrar al proyecto.

8. ¿Qué riesgos existen al sobrescribir una rama remota?

Cuando se hace git push --force se reescribe el historial de la rama en GitHub. Si otra persona ya había descargado esa rama, su versión va a quedar desactualizada y puede perder trabajo o generar conflictos difíciles de resolver. Por eso solo se debe usar en ramas propias y con cuidado.

9. ¿Qué buenas prácticas aplicaría en un proyecto real?

Siempre haría git pull antes de empezar a trabajar. Crearía una rama nueva para cada tarea. Escribiría mensajes de commit claros desde el principio para no tener que limpiarlos después. Antes de crear un Pull Request me aseguraría de que mi rama esté actualizada y el historial ordenado. Y resolvería los conflictos leyendo bien las dos versiones antes de decidir.

10. ¿Qué errores cometí durante el taller y cómo los solucioné?

Al principio intenté usar touch en PowerShell y no funcionó, tuve que cambiarme a Git Bash. También al principio copié y pegué los comandos y me aparecían caracteres raros que hacían que Git no los reconociera, aprendí que era mejor escribirlos a mano. Cuando hice el rebase interactivo el editor se cerró solo y el mensaje del commit no quedó como quería, lo resolví usando git commit --amend para cambiarlo después.

CAPTURA DEL PULL REQUEST CREADO

This screenshot shows a GitHub Pull Request (PR) titled "docs: Se agrega guia de instalacion #1". The PR is created by Jesus-Marquez03 and is ready to be merged into the main branch. The interface includes a navigation bar at the top with links to Code, Issues, Pull requests, Agents, Actions, Projects, Wiki, Security and quality, Insights, and Settings. Below the navigation bar, the PR title is displayed along with a green "Ready to merge" badge and a "Code" button. A summary bar indicates that 7 commits from the "instalacion" branch are being merged into "main". The main content area is divided into three sections: "Descripción", "Archivos modificados", and "Cambios principales". The "Descripción" section contains a list of items to be added, including a guide for installing EduCampus LMS. The "Archivos modificados" section lists the files that will be changed, specifically "docs/instalacion.md". The "Cambios principales" section details the changes, such as adding environment requirements, database configuration, and initial system verification. The "Conflictos resueltos" section shows that there are no conflicts. The "Observaciones" section lists the updates made to the develop branch and the clean history before the PR. On the right side, there are sections for "Reviewers", "Assignees", "Labels", "Projects", "Milestone", "Development", and "Notifications". The "Reviewers" section shows "No reviews". The "Assignees" section shows "No one—assign yourself". The "Labels" section shows "None yet". The "Projects" section shows "None yet". The "Milestone" section shows "No milestone". The "Development" section shows "Successfully merging this pull request may close these issues." and "None yet". The "Notifications" section shows "Unsubscribe" and "You're receiving notifications because you authored the thread." The bottom of the page shows a Windows taskbar with various application icons and the system clock.

docs: Se agrega guia de instalacion #1

Ready to merge Code

Jesus-Marquez03 wants to merge 7 commits into main from instalacion

Conversation 0 Commits 7 Checks 0 Files changed 6

Jesus-Marquez03 commented 18 minutes ago

Descripción

Se agrega documentación relacionada con:

- Guía de instalación de EduCampus LMS

Archivos modificados

- docs/instalacion.md

Cambios principales

- Requisitos del entorno
- Configuración de base de datos
- Variables de entorno
- Pasos para ejecutar el proyecto localmente
- Verificación inicial del sistema

Conflictos resueltos

- Ninguno

Observaciones

- Rama actualizada respecto a develop
- Historial limpio antes del PR

Reviewers: No reviews

Assignees: No one—assign yourself

Labels: None yet

Projects: None yet

Milestone: No milestone

Development: Successfully merging this pull request may close these issues. None yet

Notifications: Unsubscribe

1 participant

CAPTURA DE LA PESTAÑA DE ARCHIVOS MODIFICADOS

This screenshot shows the "Files changed" tab of the same GitHub Pull Request. The left sidebar displays a file explorer view of the repository, highlighting the "docs" folder and its contents: "buenas-practicas.md", "contribucion.md", "evaluaciones.md", "instalacion.md", "modulos.md", and "README.md". The main area shows the diff for "docs/buenas-practicas.md", which is a new file. The diff content includes a list of guidelines for working with Git and GitHub, such as "Acá se recogen algunas buenas prácticas para trabajar con Git y GitHub", "Haz commits pequeños, cada uno con un solo propósito.", "Crea una rama nueva para cada tarea o funcionalidad.", and "Lee bien los cambios antes de aprobar un Pull Request." The right sidebar shows the "Submit review" button and the "Viewed" status.

docs: Se agrega guia de instalacion #1

Ready to merge Code Preview

Jesus-Marquez03 wants to merge 7 commits into main from instalacion

Conversation 0 Commits 7 Checks 0 Files changed 6

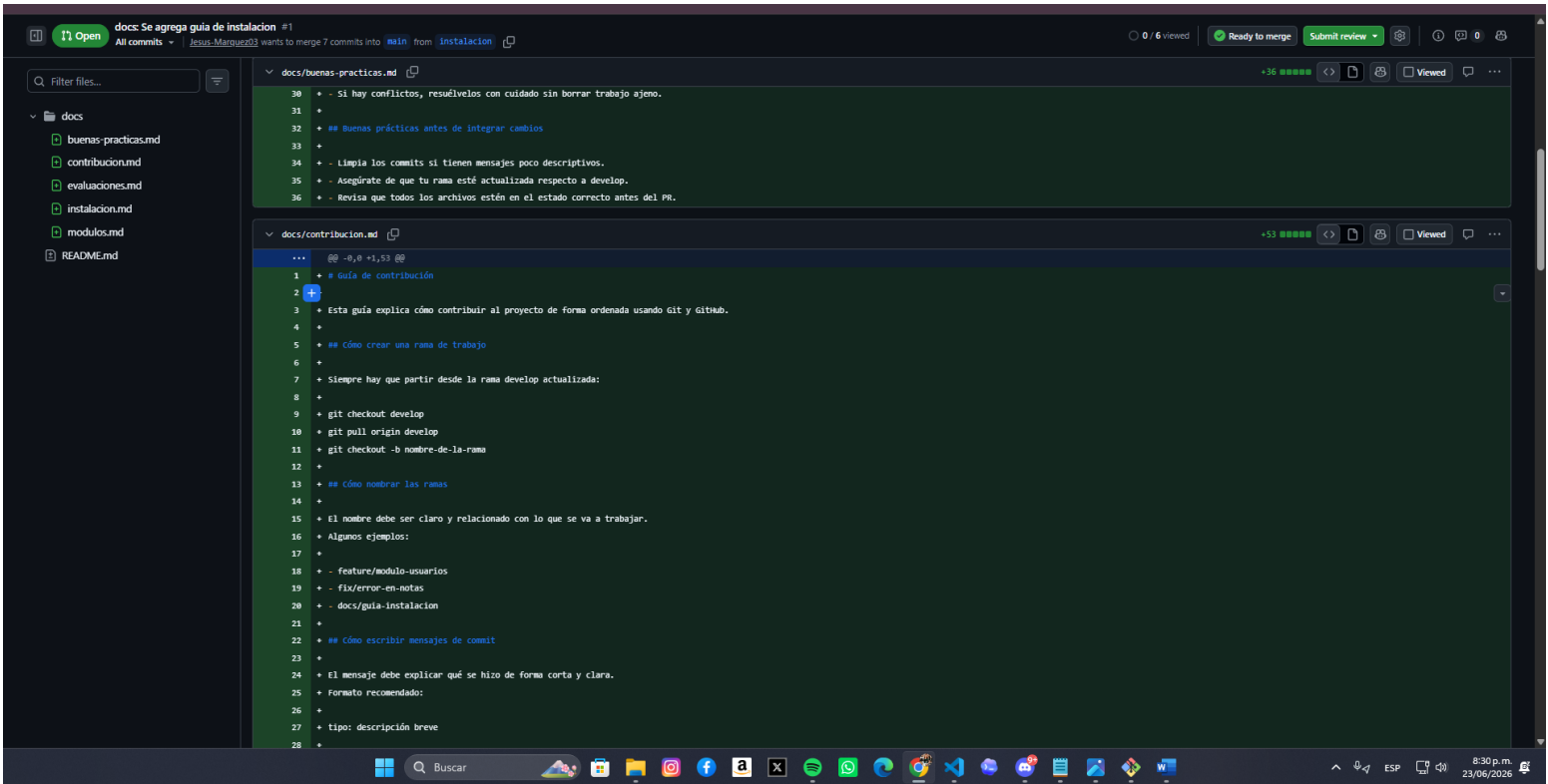
All commits

Filter files...

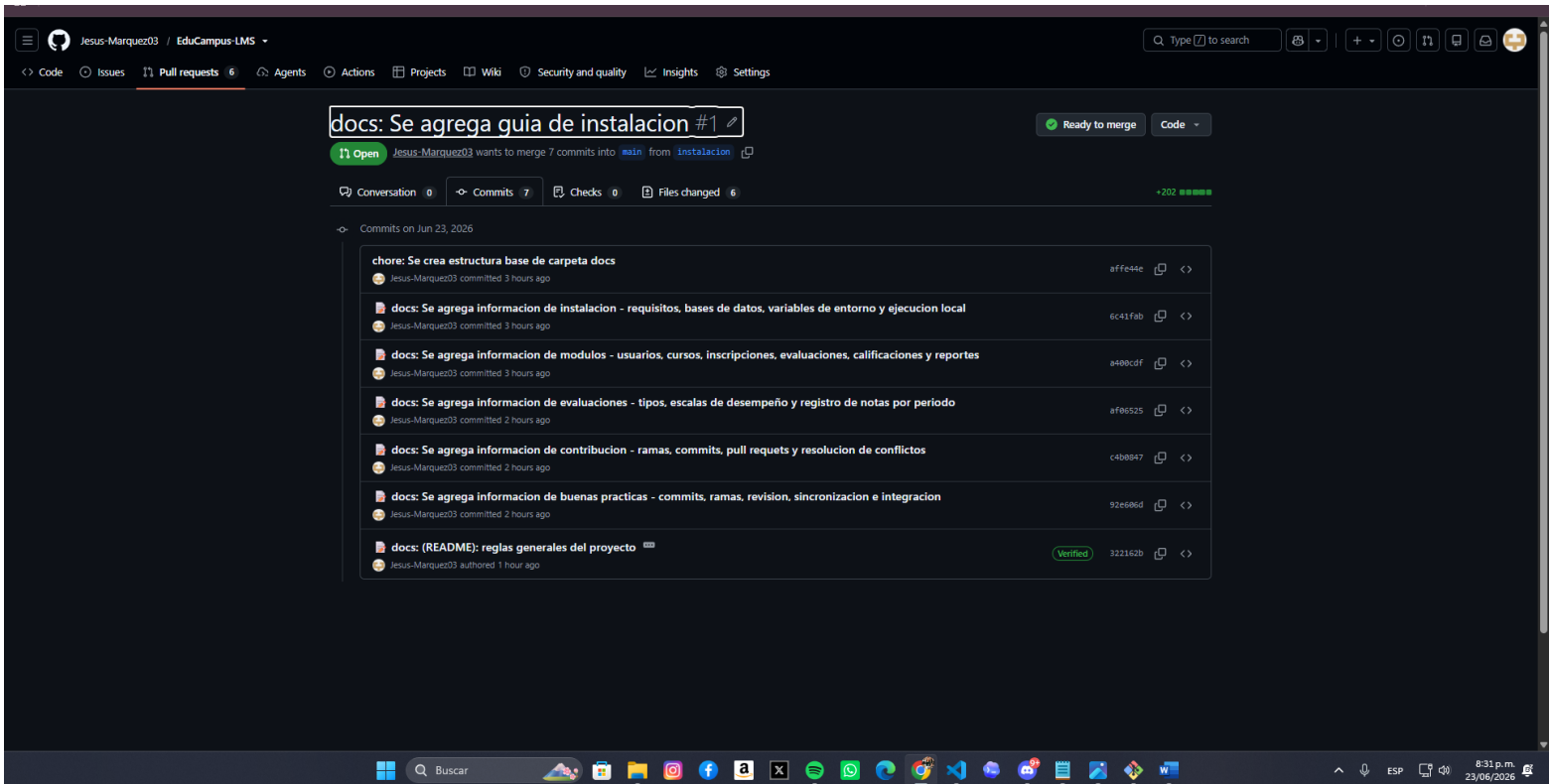
docs/buenas-practicas.md

```
+++ @@ -0,0 +1,36 @@
1 + # Buenas prácticas
2 +
3 + Acá se recogen algunas buenas prácticas para trabajar con Git y GitHub
4 + de forma ordenada dentro del proyecto.
5 +
6 + ## Buenas prácticas de commits
7 +
8 + - Haz commits pequeños, cada uno con un solo propósito.
9 + - El mensaje debe decir claramente qué se hizo.
10 + - No mezcles cambios de temas distintos en un mismo commit.
11 + - Antes de hacer git add, revisa qué archivos vas a incluir.
12 +
13 + ## Buenas prácticas de ramas
14 +
15 + - No trabajes directamente sobre main ni sobre develop.
16 + - Crea una rama nueva para cada tarea o funcionalidad.
17 + - Cuando termines y hagas merge, elimina la rama.
18 + - El nombre de la rama debe describir lo que hace.
19 +
20 + ## Buenas prácticas de revisión de código
21 +
22 + - Lee bien los cambios antes de aprobar un Pull Request.
23 + - Si algo no está claro, comenta y pregunta, no rechaces sin explicar.
24 + - Verifica que los cambios no rompan lo que ya estaba funcionando.
25 +
```

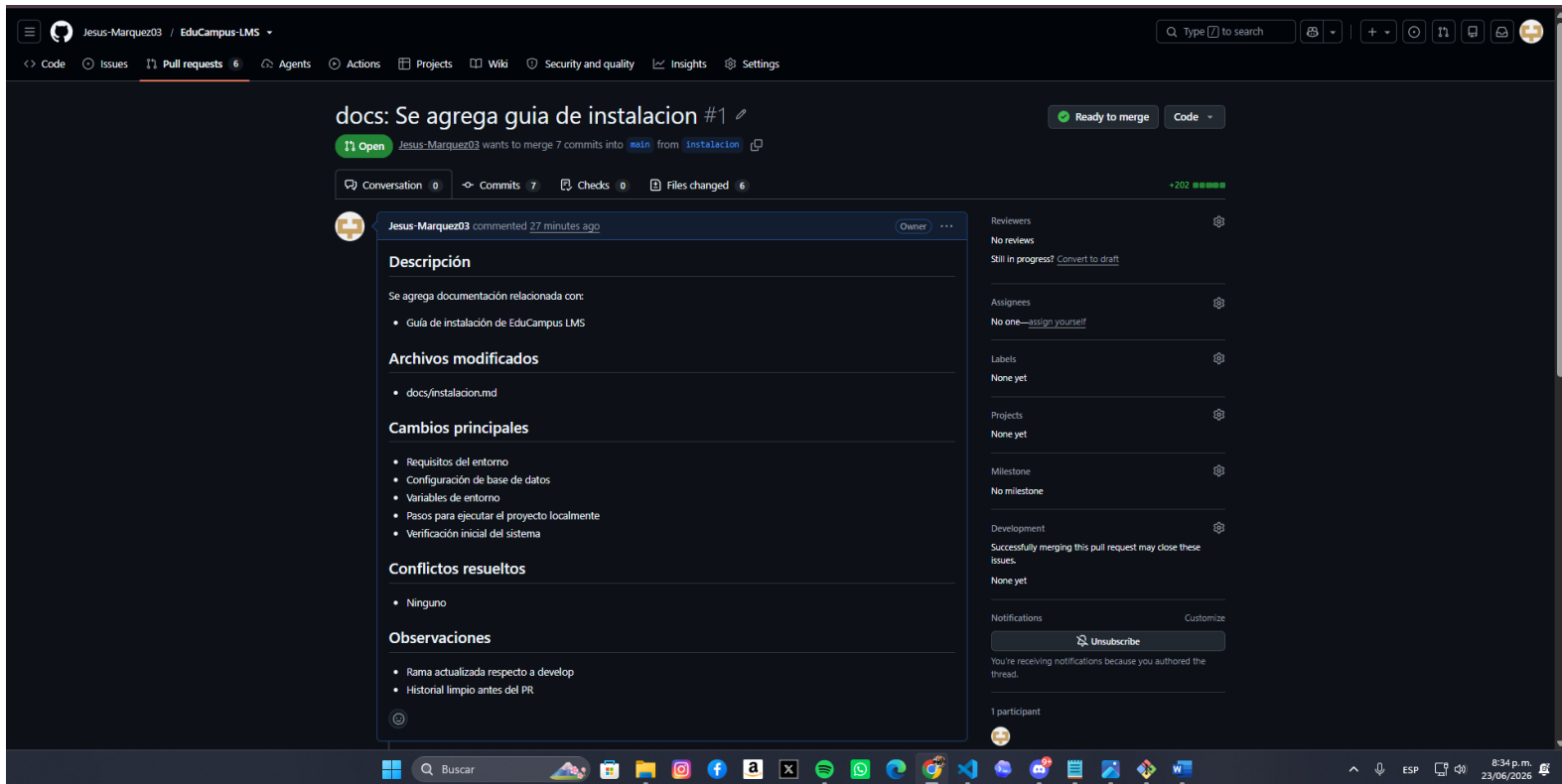
0 / 6 viewed Submit review Viewed



CAPTURA DE LOS COMMITS INCLUIDOS



CAPTURA DE LA REVISION O APROBACION



EXPLICACION DEL FLUJO REALIZADO DESDE LA CREACION DE LA RAMA HASTA EL PULL REQUEST

El proceso comenzó asegurándonos de estar siempre en la rama develop, que es desde donde se crea todo el trabajo. Antes de crear cada rama hicimos git checkout develop para confirmar que estábamos en el lugar correcto.

Luego creamos cada rama con el comando git checkout -b nombre-de-la-rama. Por ejemplo, para la guía de instalación usamos git checkout -b instalacion. Esto creó una rama nueva basada en develop y nos movió automáticamente a ella.

Una vez dentro de la rama, abrimos el archivo correspondiente en VS Code y escribimos todo el contenido que le pertenecía a esa rama. Por ejemplo en la rama instalacion escribimos los requisitos, la base de datos, las variables de entorno y los pasos para ejecutar el proyecto.

Cuando terminamos de escribir el contenido, guardamos el archivo y volvimos a Git Bash para registrar los cambios. Primero usamos git add para preparar el archivo, luego git commit -m con un mensaje descriptivo que explicaba qué se había agregado, y finalmente git push origin nombre-de-la-rama para subir la rama a GitHub.

Después de subir cada rama hicimos el merge hacia develop usando git checkout develop y luego git merge nombre-de-la-rama. Esto integró los cambios de cada rama dentro de develop y luego subimos esos cambios con git push origin develop.

Al final del taller, cuando todas las ramas estaban listas y con su contenido completo, creamos los Pull Requests desde GitHub. Para cada rama fuimos a la sección de Pull Requests, seleccionamos main como base y la rama correspondiente como compare, escribimos un título claro, una descripción detallada con los archivos modificados y los cambios principales, y creamos el PR. Esto dejó evidencia formal en GitHub de todo el trabajo realizado en cada rama.

CAPTURA HISTORIAL ANTES Y DESPUES DE REORGANIZAR LA RAMA

```
Jesus@DESKTOP-RMUCD21 MINGW64 ~/Desktop/Taller-educampus-lms (buenas-practicas-estable)
git log --oneLine
6e9f636 (HEAD -> buenas-practicas-estable, origin/buenas-practicas-estable) docs: Se agrega nuevas practicas para mensaje de commit
2321d85 (origin/develop, develop) fix: Se resuelve conflicto en README integrando ambas versiones
49c9341 (memo: docs: (README): descripcion modificada sobre EduCampus LMS
4a5a87d docs: Se actualiza descripcion del proyecto en README
322162b (origin/instalacion, instalacion) (memo: docs: (README): reglas generales del proyecto
82e606d (origin/buenas-practicas, buenas-practicas) (memo: docs: Se agrega informacion de buenas practicas - commits, ramas, revision, sincronizacion e integracion
44b0847 (memo: docs: Se agrega informacion de contribucion - ramas, commits, pull requests y resolucion de conflictos
4f0632b (origin/conclusiones, conclusiones) (memo: docs: Se agrega informacion de evaluaciones - tipos, escalas de desempeño y registro de notas por periodo
4400cdf (origin/modulos, modulos) (memo: docs: Se agrega informacion de modulos - usuarios, cursos, inscripciones, evaluaciones, calificaciones y reportes
6c41fab (memo: docs: Se agrega informacion de instalacion - requisitos, bases de datos, variables de entorno y ejecucion local
d7f644e chore: Se crea estructura base de carpeta docs
4ee049 (origin/main, main) first commit: Se agrega README.md inicial
```

Para la captura del "antes" no la pude tomar ya que el rebase se realizó sobre la rama instalación.

EVIDENCIA DE CONFLICTO

Jesus-Marquez03 / EduCampus-LMS

Code Issues Pull requests 6 Agents Actions Projects Wiki Security and quality Insights Settings

docs: Se agrega buenas prácticas para mensajes de commit #6

Open Jesus-Marquez03 wants to merge 11 commits into main from buenas-practicas-estable

Conversation Commits 11 Checks 0 Files changed 6

Jesus-Marquez03 commented 37 minutes ago

Owner

Descripción

Se agrega documentación relacionada con:

- Buenas prácticas para mensajes de commit obtenida mediante cherry-pick

Archivos modificados

- docs/buenas-practicas.md

Cambios principales

- Buenas prácticas para mensajes de commit

Conflictos resueltos

- Se resolvió conflicto al aplicar cherry-pick desde la rama experimental/buenas-practicas-avanzadas

Observaciones

- Solo se incorporó el commit aprobado por el líder técnico
- Los otros dos cambios experimentales no fueron incluidos
- Historial limpio antes del PR

Reviews

No reviews

Still in progress! Convert to draft

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Unsubscribe

You're receiving notifications because you authored the thread.

1 participant

EVIDENCIA DE INCORPORACION DE UN CAMBIO ESPECIFICO

docs: Se agrega buenas prácticas para mensajes de commit #6

Jesus-Marquez03 wants to merge 11 commits into main from buenas-practicas-estable

Conversation 0 Commits 11 Checks 0 Files changed 6

Commits on Jun 23, 2026

- chore: Se crea estructura base de carpeta docs
Jesus-Marquez03 committed 3 hours ago
- docs: Se agrega informacion de instalacion - requisitos, bases de datos, variables de entorno y ejecucion local
Jesus-Marquez03 committed 3 hours ago
- docs: Se agrega informacion de modulos - usuarios, cursos, inscripciones, evaluaciones, calificaciones y reportes
Jesus-Marquez03 committed 3 hours ago
- docs: Se agrega informacion de evaluaciones - tipos, escalas de desempeño y registro de notas por periodo
Jesus-Marquez03 committed 3 hours ago
- docs: Se agrega informacion de contribucion - ramas, commits, pull requests y resolucion de conflictos
Jesus-Marquez03 committed 3 hours ago
- docs: Se agrega informacion de buenas practicas - commits, ramas, revision, sincronizacion e integracion
Jesus-Marquez03 committed 3 hours ago
- docs: (README): reglas generales del proyecto
Jesus-Marquez03 authored 2 hours ago
- docs: Se actualiza descripcion del proyecto en README
Jesus-Marquez03 committed 2 hours ago
- docs: (README): descripcion modificada sobre EduCampus LMS
Jesus-Marquez03 authored 1 hour ago
- fix: Se resuelve conflicto en README integrando ambas versiones
Jesus-Marquez03 committed 1 hour ago
- docs: Se agrega nuevas practicas para mensaje de commit
Jesus-Marquez03 committed 1 hour ago

EVIDENCIA DE SOLUCION DEL CONFLICTO

docs: Se agrega buenas prácticas para mensajes de commit #6

Jesus-Marquez03 wants to merge 11 commits into main from buenas-practicas-estable

Conversation 0 Commits 11 Checks 0 Files changed 6

Jesus-Marquez03 commented 45 minutes ago

Descripción

Se agrega documentación relacionada con:

- Buenas prácticas para mensajes de commit obtenida mediante cherry-pick

Archivos modificados

- docs/buenas-practicas.md

Cambios principales

- Buenas prácticas para mensajes de commit

Conflictos resueltos

- Se resolvió conflicto al aplicar cherry-pick desde la rama experimental/buenas-practicas-avanzadas

Observaciones

- Solo se incorporó el commit aprobado por el líder técnico
- Los otros dos cambios experimentales no fueron incluidos
- Historial limpio antes del PR

Jesus-Marquez03 added 11 commits 3 hours ago

JUSTIFICACION DE LAS DECISIONES TOMADAS

Decidimos trabajar siempre desde la rama `develop` y no desde `main` porque `main` debe conservarse limpia y estable. Cualquier error que se cometa en el proceso de desarrollo no debe afectar la rama principal del proyecto.

Creamos una rama por cada archivo de documentación porque así cada cambio queda aislado y es más fácil rastrearlo. Si algo sale mal en una rama, no afecta el trabajo de las demás.

Usamos `rebase` en vez de `merge` para reorganizar la rama `instalacion` porque queríamos que el historial quedara lineal y limpio, sin commits de unión innecesarios. Esto hace que sea más fácil leer el historial del proyecto.

Decidimos usar `squash` en la rama `contribucion` para limpiar los commits desordenados porque antes del Pull Request el historial debe ser claro y descriptivo. Un revisor no debería ver mensajes como "arreglo" o "cambio inicial" en el historial final.

Usamos `cherry-pick` para traer solo el commit aprobado desde la rama experimental porque los otros dos cambios todavía no estaban listos para integrarse al proyecto. Era mejor traer solo lo que estaba aprobado y dejar el resto para después.

DESCRIPCION DE LOS PROBLEMAS ENCONTRADOS

El primer problema que tuve fue al intentar crear los archivos desde PowerShell. Los comandos `type nul` y `touch` no funcionaron correctamente en esa terminal. Lo resolví cambiándome a Git Bash, que es la terminal recomendada para trabajar con Git en Windows.

El segundo problema fue al copiar y pegar comandos desde el chat. Los comandos llegaban con caracteres invisibles al principio que hacían que Git no los reconociera y mostraba errores como `command not found`. Lo resolví escribiendo todos los comandos a mano directamente en Git Bash.

El tercer problema fue al hacer el `rebase` interactivo para limpiar los commits. El editor se cerró solo sin dejarme escribir el mensaje final del commit. Lo resolví usando `git commit --amend` para cambiar el mensaje del commit después de que el `squash` ya se había completado.

El cuarto problema fue al intentar crear el Pull Request de la rama `instalacion` hacia `develop`. GitHub mostraba "There isn't anything to compare" porque esa rama ya había sido mergeada a `develop` anteriormente. Lo resolví cambiando el base de `develop` a `main`, que sí tenía diferencias con las ramas de trabajo.

CONCLUSIONES

Durante este taller aprendí que Git no es solo para guardar archivos, sino que es una herramienta que ayuda a organizar el trabajo en equipo de forma ordenada y segura. Cada comando tiene un propósito específico y usarlo en el momento correcto hace que el proyecto sea más fácil de mantener.

Entendí que trabajar en ramas separadas es fundamental porque permite que cada persona trabaje en su parte sin afectar el trabajo de los demás. Si algo sale mal, se puede corregir en la rama sin dañar el proyecto principal.

También aprendí que los conflictos no son un error grave, sino algo normal que pasa cuando dos personas modifican el mismo archivo. Lo importante es resolverlos con cuidado, leyendo bien las dos versiones antes de decidir cuál queda.

Finalmente entendí la importancia de mantener el historial de commits limpio y ordenado. Un buen mensaje de commit puede ahorrarle mucho tiempo a cualquier persona que necesite entender qué cambios se hicieron y por qué.

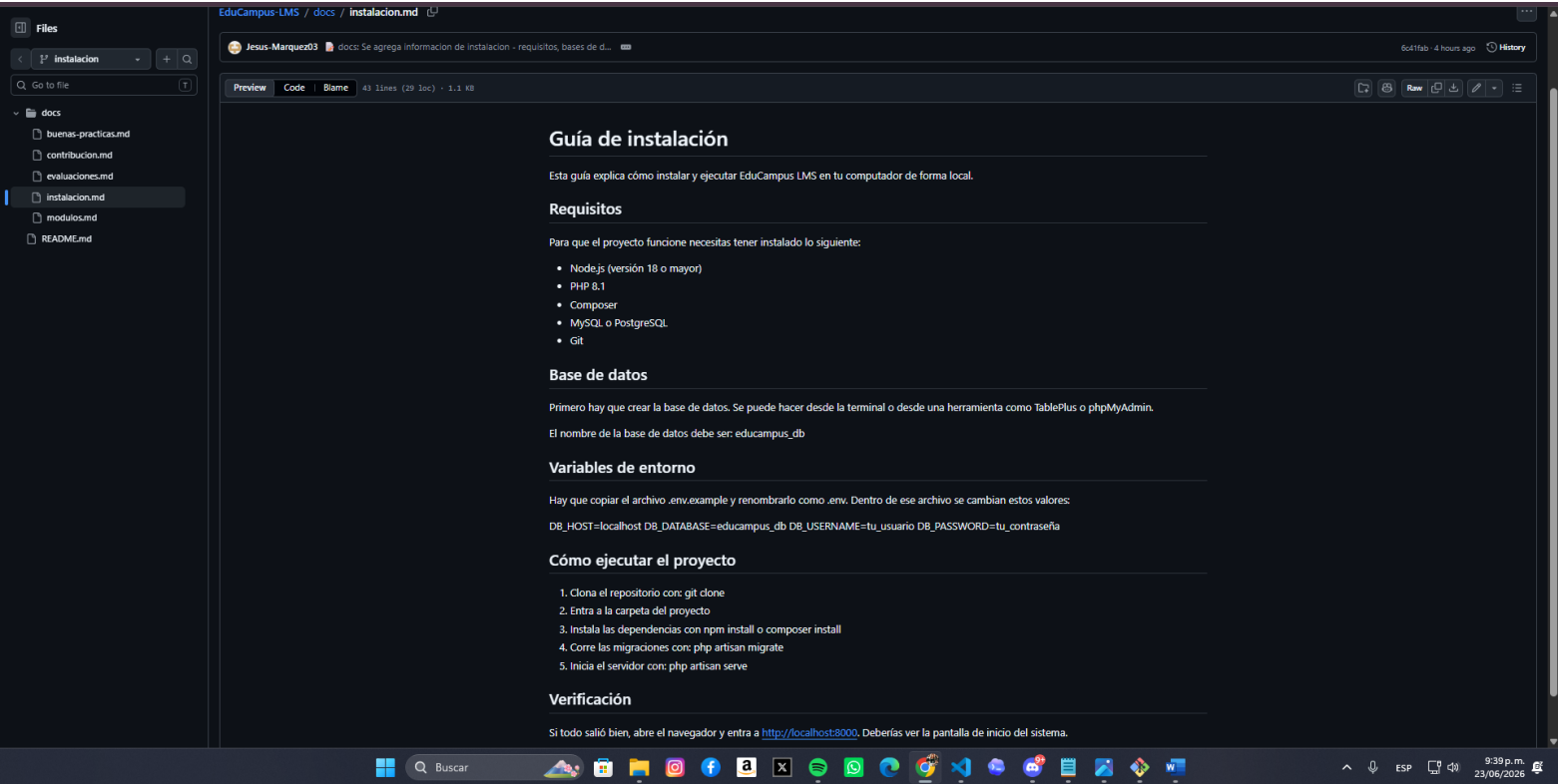
RETO 1 EVIDENCIAS ESPERADAS:

Captura de la rama creada en GITHUB

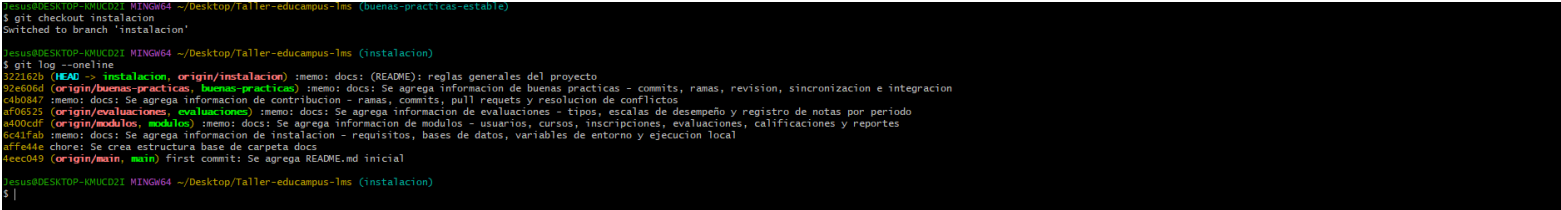
The screenshot shows the GitHub interface for a repository. The 'Branches' tab is selected, displaying a list of branches. The 'main' branch is highlighted as the default. Below it, a message states 'Your main branch isn't protected'. The 'Your branches' section lists several branches with their update times, check status, and pull request links. The 'Active branches' section shows the same list of branches.

Branch	Updated	Check status	Behind / Ahead	Pull request
main	3 hours ago		Default	
buena-practica-estable	1 hour ago		1 11	#6
experimental/buena-practica-avanzada	1 hour ago		1 13	
contribucion	1 hour ago		1 6	#4
develop	2 hours ago		1 10	
instalacion	2 hours ago		1 7	#1

Captura del archivo modificado:



Captura del historial de cambios de la rama:



Explicación breve de qué cambios fueron publicados

Durante el Reto 1 se crearon cinco ramas de trabajo, una por cada archivo de documentación del proyecto. En cada rama se agregó el contenido correspondiente a su archivo: la guía de instalación, los módulos del sistema, las evaluaciones académicas, la guía de contribución y las buenas prácticas. Cada archivo fue guardado con un commit descriptivo y subido a GitHub con git push. Luego cada rama fue integrada a develop mediante git merge.

Pregunta de reflexión — ¿Por qué no es recomendable trabajar directamente sobre la rama principal?

Porque la rama principal es la versión estable del proyecto que todos usan como base. Si alguien trabaja directamente ahí y comete un error, ese error afecta a todo el equipo y es más difícil de corregir. Trabajando en ramas separadas cada persona puede hacer cambios sin arriesgar el trabajo de los demás, y solo se integra a la rama principal cuando el trabajo ya está revisado y aprobado.

RETO 2 EVIDENCIAS ESPERADAS:

Captura del nuevo contenido del README

The screenshot shows a GitHub repository for 'EduCampus LMS'. The README file is open, displaying the following content:

EduCampus LMS

EduCampus LMS es una solución web académica diseñada para centralizar la gestión institucional, los cursos, usuarios, evaluaciones, calificaciones y reportes académicos.

Objetivo

Centralizar los procesos académicos de una institución educativa mediante una solución web modular.

Estado del proyecto

Documentación inicial en construcción.

Reglas generales del proyecto

1. Todo cambio debe realizarse en una rama independiente.
2. Todo aporte debe ser revisado antes de integrarse.
3. Los mensajes de commit deben ser claros y descriptivos.
4. Los conflictos deben resolverse de forma consciente.

Historial actualizado

```
jesus@DESKTOP-KMUCD21 MINGW64 ~/Desktop/Taller-educampus-lms (instalacion)
$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.

jesus@DESKTOP-KMUCD21 MINGW64 ~/Desktop/Taller-educampus-lms (develop)
$ git log --oneline
2321d85 (HEAD -> develop, origin/develop) fix: Se resuelve conflicto en README integrando ambas versiones
85c9341 :memo: docs: (README): description modificada sobre EduCampus LMS
8a5a82d docs: Se actualiza descripción del proyecto en README
322162b (origin/instalacion, instalacion) :memo: docs: (README): reglas generales del proyecto
92e806d (origin/buenas-practicas, buenas-practicas) :memo: docs: Se agrega informacion de buenas practicas - commits, ramas, revision, sincronizacion e integracion
a1b0847 :memo: docs: Se agrega informacion de contribucion - ramas, commits, pull requests y resolución de conflictos
af06525 (origin/evaluaciones, evaluaciones) :memo: docs: Se agrega informacion de evaluaciones - tipos, escalas de desempeño y registro de notas por periodo
a400cdd (origin/modulos, modulos) :memo: docs: Se agrega informacion de modulos - usuarios, cursos, inscripciones, evaluaciones, calificaciones y reportes
0c41fab :memo: docs: Se agrega informacion de instalacion - requisitos, bases de datos, variables de entorno y ejecucion local
af9e44c chore: Se crea estructura base de carpeta docs
4e0c049 (origin/main, main) first commit: Se agrega README.md inicial

jesus@DESKTOP-KMUCD21 MINGW64 ~/Desktop/Taller-educampus-lms (develop)
$
```

Explicación de qué ocurrió al traer los cambios remotos

Al ejecutar `git pull origin develop` Git descargó automáticamente los cambios que el líder técnico había hecho en GitHub y los unió con el trabajo local. El archivo `README.md` quedó actualizado con la nueva sección de reglas generales y el trabajo que ya estaba hecho localmente no se perdió.

Pregunta de reflexión — ¿Qué riesgo existe si un desarrollador trabaja varios días sin actualizar su repositorio local?

Si pasan varios días sin hacer `git pull`, el repositorio local va quedando cada vez más desactualizado respecto al remoto. Cuando finalmente intente subir sus cambios, es muy probable que aparezcan muchos conflictos porque otras personas habrán modificado los mismos archivos. Entre más tiempo pase sin sincronizarse, más difícil será resolver esos conflictos.

RETO 3 EVIDENCIAS ESPERADAS:

Historial antes de reorganizar

```
jesus@DESKTOP-KNUK021 MINGW64 ~/Desktop/Taller-educampus-lms (buenas-practicas)
$ git push origin buenas-practicas
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), 1.07 MiB/s, done.
Writing objects: 100% (4/4), 1.07 KiB | 1.07 MiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'buenas-practicas' on GitHub by visiting:
remote:   https://github.com/Jesus-Marquez03/EduCampus-LMS/pull/new/buenas-practicas
remote:
To https://github.com/Jesus-Marquez03/EduCampus-LMS.git
 * [new branch]   buenas-practicas -> buenas-practicas

jesus@DESKTOP-KNUK021 MINGW64 ~/Desktop/Taller-educampus-lms (buenas-practicas)
$ git checkout develop
bash: '$'\302\203git': command not found

jesus@DESKTOP-KNUK021 MINGW64 ~/Desktop/Taller-educampus-lms (buenas-practicas)
$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.

jesus@DESKTOP-KNUK021 MINGW64 ~/Desktop/Taller-educampus-lms (develop)
$ git checkout develop
Already on 'develop'
Your branch is up to date with 'origin/develop'.

jesus@DESKTOP-KNUK021 MINGW64 ~/Desktop/Taller-educampus-lms (develop)
$ git merge buenas-practicas
Updating c4b0847..92e606d
Fast-forward
 docs/buenas-practicas.md | 36 +++++++++++++++++++++++++++++++++++++
 1 file changed, 36 insertions(+)

jesus@DESKTOP-KNUK021 MINGW64 ~/Desktop/Taller-educampus-lms (develop)
$ git push origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Jesus-Marquez03/EduCampus-LMS.git
   c4b0847..92e606d  develop -> develop

jesus@DESKTOP-KNUK021 MINGW64 ~/Desktop/Taller-educampus-lms (develop)
$ git pull origin develop
From https://github.com/Jesus-Marquez03/EduCampus-LMS
 * branch      develop      -> FETCH_HEAD
Already up to date.

jesus@DESKTOP-KNUK021 MINGW64 ~/Desktop/Taller-educampus-lms (develop)
$ git pull origin develop
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 1.30 KiB | 102.00 KiB/s, done.
From https://github.com/Jesus-Marquez03/EduCampus-LMS
 * branch      develop      -> FETCH_HEAD
   92e606d..322162b  develop -> origin/develop
Updating 92e606d..322162b
Fast-forward
 README.md | Bin 834 -> 600 bytes
 1 file changed, 0 insertions(+), 0 deletions(-)

jesus@DESKTOP-KNUK021 MINGW64 ~/Desktop/Taller-educampus-lms (develop)
$ git checkout instalacion
Switched to branch 'instalacion'

jesus@DESKTOP-KNUK021 MINGW64 ~/Desktop/Taller-educampus-lms (instalacion)
$ git rebase develop
Successfully rebased and updated refs/heads/instalacion.
```

Captura del historial despues de reorganizar la rama

```
Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (develop)
$ git checkout instalacion
Switched to branch 'instalacion'

Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (instalacion)
$ git log --oneline
322162b (HEAD -> instalacion, origin/instalacion) :memo: docs: (README): reglas generales del proyecto
92e606d (origin/buenas-practicas, buenas-practicas) :memo: docs: Se agrega información de buenas prácticas - comits, ramas, revision, sincronizacion e integracion
c4b0847 :memo: docs: Se agrega información de contribucion - ramas, commits, pull requests y resolucion de conflictos
af06525 (origin/evaluaciones, evaluaciones) :memo: docs: Se agrega información de evaluaciones - tipos, escalas de desempeño y registro de notas por periodo
a400cdf (origin/modulos, modulos) :memo: docs: Se agrega información de modulos - usuarios, cursos, inscripciones, evaluaciones, calificaciones y reportes
6c41fab :memo: docs: Se agrega información de instalacion - requisitos, bases de datos, variables de entorno y ejecucion local
affe4de chore: Se crea estructura base de carpeta docs
4eec049 (origin/main, main) First commit: Se agrega README.md inicial
```

```
Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (instalacion)
$
```

Explicación de qué cambió en el historial

Antes del rebase la rama instalacion estaba basada en una versión antigua de develop. Después del rebase los commits de la rama quedaron reubicados encima de los commits más recientes de develop, como si la rama hubiera sido creada desde la versión más nueva del proyecto. El historial quedó lineal y limpio.

¿Hubo conflictos?

No hubo conflictos. El rebase se completó automáticamente sin necesidad de intervención manual.

Pregunta de reflexión — ¿Qué diferencia hay entre merge y rebase?

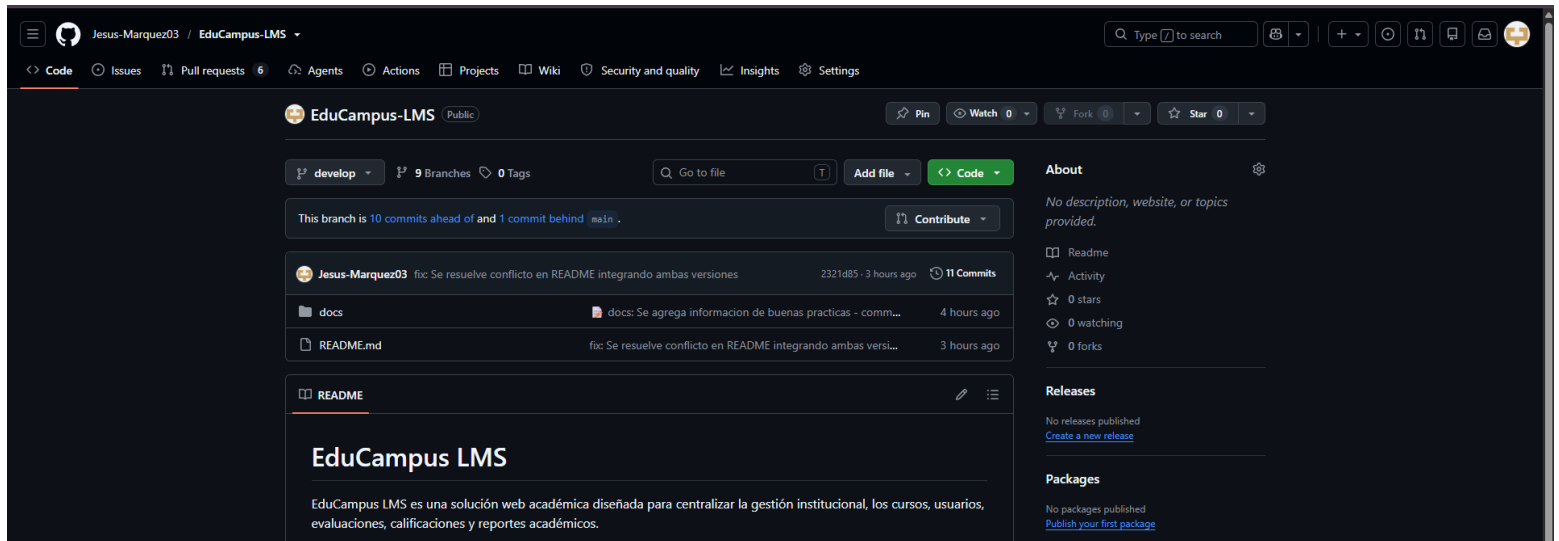
Con merge se crea un commit extra que une las dos ramas y el historial muestra las dos líneas de trabajo por separado. Con rebase los commits de la rama se mueven encima de la rama base y el historial queda como una sola línea continua, más fácil de leer y entender.

RETO 4 EVIDENCIAS ESPERADAS:

Captura del conflicto

```
Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (develop)
$ git pull origin develop
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 1.13 KiB | 88.00 KiB/s, done.
From https://github.com/Jesus-Marquez03/EduCampus-LMS
* branch          develop      -> FETCH_HEAD
   322162b..c9c9341 develop      -> origin/develop
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

Captura solución aplicada



Estado limpio del repositorio

```
Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (develop|MERGING)
$ git commit -m "fix: Se resuelve conflicto en README integrando ambas versiones"
[develop 2321d85] fix: Se resuelve conflicto en README integrando ambas versiones
```

Historial después de resolver

```
Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (instalacion)
$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.

Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (develop)
$ git log --oneline
2321d85 (HEAD -> develop, origin/develop) fix: Se resuelve conflicto en README integrando ambas versiones
c9c9341 :memo: docs: (README): descripcion modificada sobre EduCampus LMS
8a5a82d docs: Se actualiza descripcion del proyecto en README
322162b (origin/instalacion, instalacion) :memo: docs: (README): reglas generales del proyecto
92e606d (origin/buenas-practicas, buenas-practicas) :memo: docs: Se agrega informacion de buenas practicas - commits, ramas, revision, sincronizacion e integracion
c4b0847 :memo: docs: Se agrega informacion de contribucion - ramas, commits, pull requets y resolucion de conflictos
af06525 (origin/evaluaciones, evaluaciones) :memo: docs: Se agrega informacion de evaluaciones - tipos, escalas de desempeño y registro de notas por periodo
a400cdf (origin/modulos, modulos) :memo: docs: Se agrega informacion de modulos - usuarios, cursos, inscripciones, evaluaciones, calificaciones y reportes
6c41fab :memo: docs: Se agrega informacion de instalacion - requisitos, bases de datos, variables de entorno y ejecucion local
affe44e chore: Se crea estructura base de carpeta docs
4eec049 (origin/main, main) first commit: Se agrega README.md inicial

Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (develop)
```

Explicación de por qué se decidió esa versión final

Se decidió integrar las dos versiones porque cada una tenía información válida. La versión del líder técnico hablaba de centralizar la gestión institucional y la versión propia mencionaba usuarios, calificaciones y reportes académicos. La versión final

combinó ambas ideas para que el README describiera el proyecto de forma más completa.

Pregunta de reflexión — ¿Por qué un conflicto no debe resolverse aceptando solo una versión sin revisar?

Porque cada versión fue escrita por una persona con una intención específica. Si se elimina una sin leerla, se puede perder información importante o decisiones que el equipo ya había tomado. Lo correcto es leer las dos versiones, entender qué quería decir cada una y construir una solución que integre lo mejor de las dos.

RETO 5 EVIDENCIAS ESPERADAS:

Historial antes de limpiar

Comentario: No la pude entregar ya que primero hice el squash, antes habían 3 commits con mensajes “cambio inicial”, “arreglo” y “mas texto”.

Historial después de limpiar

```
Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (develop)
$ git checkout contribucion
Switched to branch 'contribucion'

Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (contribucion)
$ git log --oneline
77ca349 (HEAD -> contribucion, origin/contribucion) Agrega guia de contribucion del proyecto
c4b0847 :memo: docs: Se agrega informacion de contribucion - ramas, commits, pull requets y resolucion de conflictos
af06525 (origin/evaluaciones, evaluaciones) :memo: docs: Se agrega informacion de evaluaciones - tipos, escalas de desempeño y registro de notas por periodo
a400cdf (origin/modulos, modulos) :memo: docs: Se agrega informacion de modulos - usuarios, cursos, inscripciones, evaluaciones, calificaciones y reportes
6c41fab :memo: docs: Se agrega informacion de instalacion - requisitos, bases de datos, variables de entorno y ejecucion local
affe44e chore: Se crea estructura base de carpeta docs
4eec049 (origin/main, main) first commit: Se agrega README.md inicial
```

Pull request con historial ordenado

The screenshot shows a GitHub pull request for the repository 'EduCampus-LMS'. The pull request is titled 'docs: Se agrega guía de contribución #4' and is in the 'Ready to merge' state. It shows 6 commits from the 'contribucion' branch to the 'main' branch. The commits are listed in chronological order, with the most recent at the top. Each commit entry includes a commit icon, the commit message, the commit hash, and the time since it was committed. The commits are:

- chore: Se crea estructura base de carpeta docs (affe44e, 5 hours ago)
- docs: Se agrega informacion de instalacion - requisitos, bases de datos, variables de entorno y ejecucion local (6c41fab, 4 hours ago)
- docs: Se agrega informacion de modulos - usuarios, cursos, inscripciones, evaluaciones, calificaciones y reportes (a400cdf, 4 hours ago)
- docs: Se agrega informacion de evaluaciones - tipos, escalas de desempeño y registro de notas por periodo (af06525, 4 hours ago)
- docs: Se agrega informacion de contribucion - ramas, commits, pull requets y resolucion de conflictos (c4b0847, 4 hours ago)
- Agrega guia de contribucion del proyecto (77ca349, 2 hours ago)

Explicación de por qué el historial final es mejor

Antes había tres commits con mensajes que no explicaban nada: "cambio inicial", "arreglo" y "mas texto". Cualquier persona que viera ese historial no entendería qué se hizo. Después del squash quedó un solo commit con un mensaje claro que describe exactamente qué contiene ese cambio.

Pregunta de reflexión — ¿Por qué es importante limpiar commits antes de un Pull Request?

Porque el Pull Request es la presentación formal del trabajo ante el equipo. Si el historial está desordenado con mensajes poco descriptivos, la persona que revisa el PR tiene que adivinar qué se hizo en cada paso. Un historial limpio hace que la revisión sea más rápida, más clara y más profesional.

RETO 6 EVIDENCIAS ESPERADAS:

Rama experimental con varios cambios y commit específico seleccionado: en esta misma captura se puede ver (2830dce docs: Se agrega nuevas practicas para mensaje de commit)

Se agrega nuevas practicas para mensaje de commit)

```
Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (contribucion)
$ git checkout experimental/buenas-practicas-avanzadas
Switched to branch 'experimental/buenas-practicas-avanzadas'

Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (experimental/buenas-practicas-avanzadas)
$ git log --oneline
4b3e9a9 (HEAD -> experimental/buenas-practicas-avanzadas, origin/experimental/buenas-practicas-avanzadas) docs: Se agrega buenas practicas para revisar pull requests
2830dce docs: Se agrega nuevas practicas para mensaje de commit
f5a4bec docs: Se agrega buenas practicas para nombrar ramas
2321d85 (origin/develop, develop) fix: Se resuelve conflicto en README integrando ambas versiones
c9c9341 :memo: docs: (README): descripcion modificada sobre EduCampus LMS
8a5a82d docs: Se actualiza descripcion del proyecto en README
322162b (origin/instalacion, instalacion) :memo: docs: (README): reglas generales del proyecto
92e606d (origin/buenas-practicas, buenas-practicas) :memo: docs: Se agrega informacion de buenas practicas - commits, ramas, revision, sincronizacion e integracion
c4b0847 :memo: docs: Se agrega informacion de contribucion - ramas, commits, pull requets y resolucion de conflictos
af06525 (origin/evaluaciones, evaluaciones) :memo: docs: Se agrega informacion de evaluaciones - tipos, escalas de desempeño y registro de notas por periodo
a400cdf (origin/modulos, modulos) :memo: docs: Se agrega informacion de modulos - usuarios, cursos, inscripciones, ev
aluaciones, calificaciones y reportes
6c41fab :memo: docs: Se agrega informacion de instalacion - requisitos, bases de datos, variables de entorno y ejecucion local
affe44e chore: Se crea estructura base de carpeta docs
4eec049 (origin/main, main) first commit: Se agrega README.md inicial
```

Rama estable con solo el cambio aprobado e historial con la incorporación puntual

```
Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (experimental/buenas-practicas-avanzadas)
$ git checkout buenas-practicas-estable
bash: '$'\302\226git': command not found

Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (experimental/buenas-practicas-avanzadas)
$ git checkout buenas-practicas-estable
Switched to branch 'buenas-practicas-estable'

Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (buenas-practicas-estable)
$ git log --oneline
9e9fc96 (HEAD -> buenas-practicas-estable, origin/buenas-practicas-estable) docs: Se agrega nuevas practicas para mensaje de commit
2321d85 (origin/develop, develop) fix: Se resuelve conflicto en README integrando ambas versiones
c9c9341 :memo: docs: (README): descripcion modificada sobre EduCampus LMS
8a5a82d docs: Se actualiza descripcion del proyecto en README
322162b (origin/instalacion, instalacion) :memo: docs: (README): reglas generales del proyecto
92e606d (origin/buenas-practicas, buenas-practicas) :memo: docs: Se agrega informacion de buenas practicas - commits, ramas, revision, sincronizacion e integracion
c4b0847 :memo: docs: Se agrega informacion de contribucion - ramas, commits, pull requets y resolucion de conflictos
af06525 (origin/evaluaciones, evaluaciones) :memo: docs: Se agrega informacion de evaluaciones - tipos, escalas de desempeño y registro de notas por periodo
a400cdf (origin/modulos, modulos) :memo: docs: Se agrega informacion de modulos - usuarios, cursos, inscripciones, evaluaciones, calificaciones y reportes
6c41fab :memo: docs: Se agrega informacion de instalacion - requisitos, bases de datos, variables de entorno y ejecucion local
affe44e chore: Se crea estructura base de carpeta docs
4eec049 (origin/main, main) first commit: Se agrega README.md inicial
```


Explicación de por qué no se integró toda la rama experimental

Porque los otros dos cambios todavía no estaban aprobados por el líder técnico. Integrar toda la rama hubiera traído cambios que no estaban listos para el proyecto. Con cherry-pick se pudo traer únicamente el cambio aprobado sin afectar el resto del trabajo.

Pregunta de reflexión — ¿En qué situaciones conviene usar cherry-pick?

Conviene usarlo cuando en una rama hay varios cambios pero solo uno o algunos están listos para integrarse al proyecto. También es útil cuando se cometió un error en una rama y se quiere llevar solo un arreglo específico a otra rama sin traer todo lo demás.

RETO 7 EVIDENCIAS ESPERADAS:

El conflicto

The screenshot shows a GitHub pull request interface for the repository 'Jesus-Marquez03 / EduCampus-LMS'. The pull request is titled 'docs: Se agrega buenas prácticas para mensajes de commit #6' and is currently open. It shows 11 commits and 6 files changed. The pull request is being merged from the 'main' branch to the 'buena-practicas-estable' branch. The interface includes a sidebar with navigation options like Code, Issues, Pull requests, Agents, Actions, Projects, Wiki, Security and quality, Insights, and Settings. The main content area displays the pull request details, including a description, a list of modified files, and a list of resolved conflicts. The description states: 'Se agrega documentación relacionada con: Buenas prácticas para mensajes de commit obtenida mediante cherry-pick'. The modified files list includes 'docs/buenas-practicas.md'. The resolved conflicts section mentions: 'Se resolvió conflicto al aplicar cherry-pick desde la rama experimental/buenas-practicas-avanzadas'. The observations section notes: 'Solo se incorporó el commit aprobado por el líder técnico', 'Los otros dos cambios experimentales no fueron incluidos', and 'Historial limpio antes del PR'. The right sidebar shows the pull request's status, including 'No reviews', 'No one assigned', 'No labels', 'No projects', 'No milestone', and 'No development'. The bottom of the interface shows the 'Merge conflicts' button and the 'Code' dropdown menu.

docs: Se agrega buenas prácticas para mensajes de commit #6

Jesus-Marquez03 wants to merge 11 commits into main from buenas-practicas-estable

Conversation 0 Commits 11 Checks 0 Files changed 6 +207

Jesus-Marquez03 commented 2 hours ago

Descripción

Se agrega documentación relacionada con:

- Buenas prácticas para mensajes de commit obtenida mediante cherry-pick

Archivos modificados

- docs/buenas-practicas.md

Cambios principales

- Buenas prácticas para mensajes de commit

Conflictos resueltos

- Se resolvió conflicto al aplicar cherry-pick desde la rama experimental/buenas-practicas-avanzadas

Observaciones

- Solo se incorporó el commit aprobado por el líder técnico
- Los otros dos cambios experimentales no fueron incluidos
- Historial limpio antes del PR

Reviews: No reviews. Still in progress? Convert to draft.

Assignees: No one—assign yourself.

Labels: None yet.

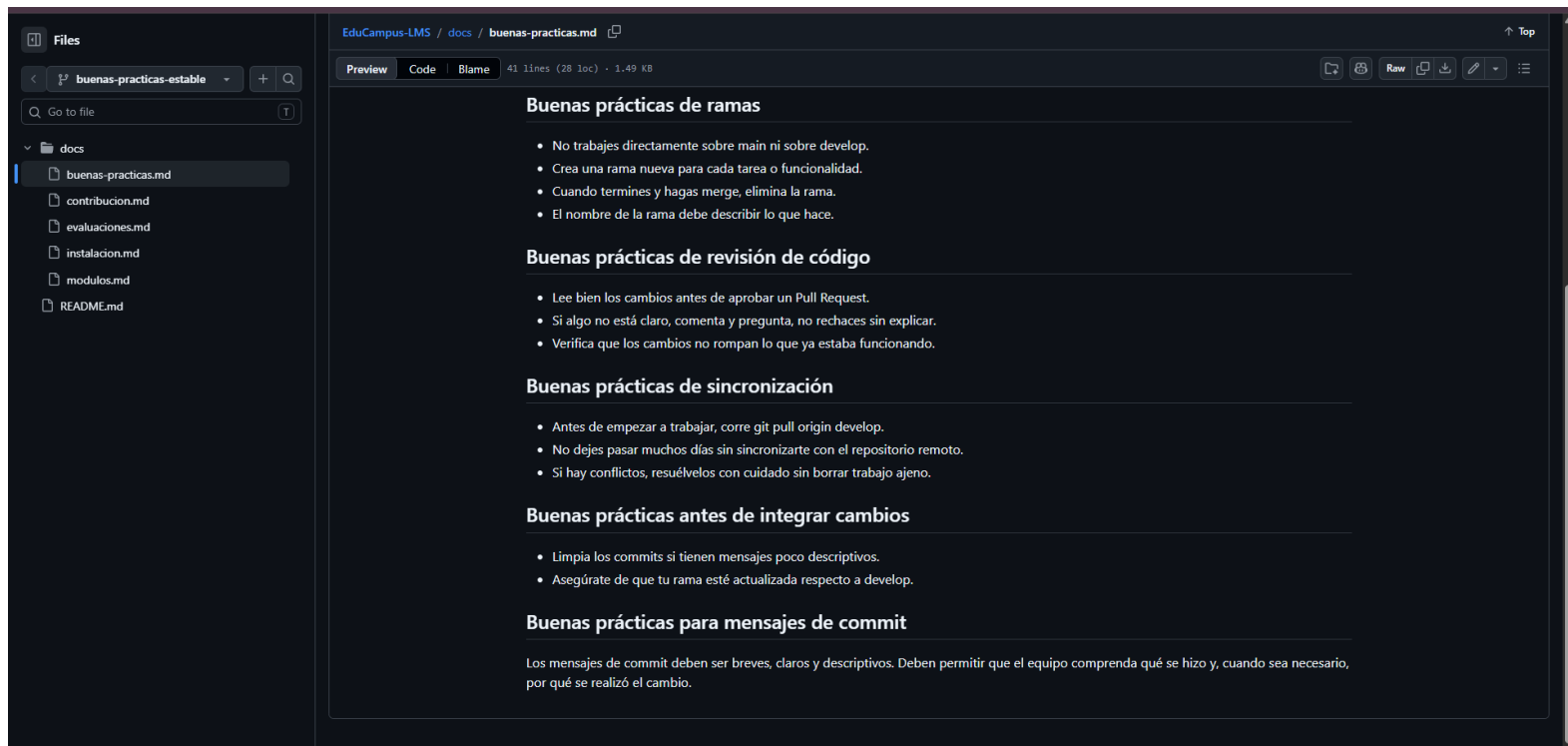
Projects: None yet.

Milestone: No milestone.

Development: Successfully merging this pull request may close these issues. None yet.

Notifications: Unsubscribe

La solución final y archivo sin duplicidad



Historial final

```
Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (buenas-practicas-estable)
$ git checkout buenas-practicas-estable
Already on 'buenas-practicas-estable'

Jesus@DESKTOP-KMUCD2I MINGW64 ~/Desktop/Taller-educampus-lms (buenas-practicas-estable)
$ git log --oneline
9e9fc96 (HEAD -> buenas-practicas-estable, origin/buenas-practicas-estable) docs: Se agrega nuevas practicas para mensaje de commit
2321d85 (origin/develop, develop) fix: Se resuelve conflicto en README integrando ambas versiones
c9c9341 :memo: docs: (README): descripcion modificada sobre EduCampus LMS
8a5a82d docs: Se actualiza descripcion del proyecto en README
322162b (origin/instalacion, instalacion) :memo: docs: (README): reglas generales del proyecto
92e606d (origin/buenas-practicas, buenas-practicas) :memo: docs: Se agrega informacion de buenas practicas - commits, ramas, revision, sincronizacion e integracion
c4b0847 :memo: docs: Se agrega informacion de contribucion - ramas, commits, pull requets y resolucion de conflictos
af06525 (origin/evaluaciones, evaluaciones) :memo: docs: Se agrega informacion de evaluaciones - tipos, escalas de desempeño y registro de notas por periodo
a400cdf (origin/modulos, modulos) :memo: docs: Se agrega informacion de modulos - usuarios, cursos, inscripciones, evaluaciones, calificaciones y reportes
6c41fab :memo: docs: Se agrega informacion de instalacion - requisitos, bases de datos, variables de entorno y ejecucion local
affe44e chore: Se crea estructura base de carpeta docs
4eec049 (origin/main, main) first commit: Se agrega README.md inicial
```

Explicación de cómo se decidió la versión final

La rama estable tenía una sección llamada "Mensajes de commit" con una descripción corta. El cherry-pick traía una versión más completa llamada "Buenas prácticas para mensajes de commit". Se decidió conservar el título más descriptivo de la versión entrante y redactar el contenido combinando las dos ideas para que quedara más completo y claro.